

Fulton Hogan Assignment

Software Modelling FINAL Report ENSE706

Fateh Bhular 23217987



Summary from Progress Report

The first phase of this assignment focused on the initial planning of the new Project Costing & Management system of Fulton Hogan. We focused on the three most important sub-departments which were **Project Management, Finance, and Operations**. Each of these departments have three important stakeholders each:



Project Management	Finance	Operations
Project Manager Project Coordinator Site Lead	Financial Controller Auditor Project Accountant	Site Foreman Heavy-Machine Operator General Labourer

During requirements elicitation, two stakeholders from each sub-department were interviewed about their current system “JD Edwards”. It was noted that each user faced issues that reduced productivity or halted work all together. After all the interviews were finished, it was determined that their existing system suffered from data latency, a lack of real-time synchronisation, and data inconsistencies.

With the interviews, and some observations of JD Edwards online, the results were translated into functional and non-function requirements. These requirements were analysed thoroughly and focused on the main issues of the existing system being data integrity, system consistency, and communication between/in departments.

During UML Requirements Modelling, the system’s behaviour was implemented using a **Use-Case Diagram**. This diagram mapped the relationships between actors-actions, and actions-actions. The final iteration of the user-case diagram was used as a blueprint for the **Class Diagram** later on, which focused on the structural frame of the system program. This diagram converted the different use-cases into concrete and abstract classes with labeled interactions. The last iteration of the class diagram was a great improvement over the first iteration, as more classes were added to support previous inconsistencies. Design principles such as SRP, ISP, etc were improved throughout the iterations, and abstract classes were also improved.

During the iterations of the use-case and class diagrams, the **C# prototype** was being produced. The initial iteration was focused on the main concrete classes that were stated in the use-case descriptions section. During production, design risks were identified, such as high coupling, low cohesion, not following SRP, and more. The Agile cycle was implemented because it was the right fit, allowing improvements to be made during development and not at the end when everything is almost finished.

[illegible]

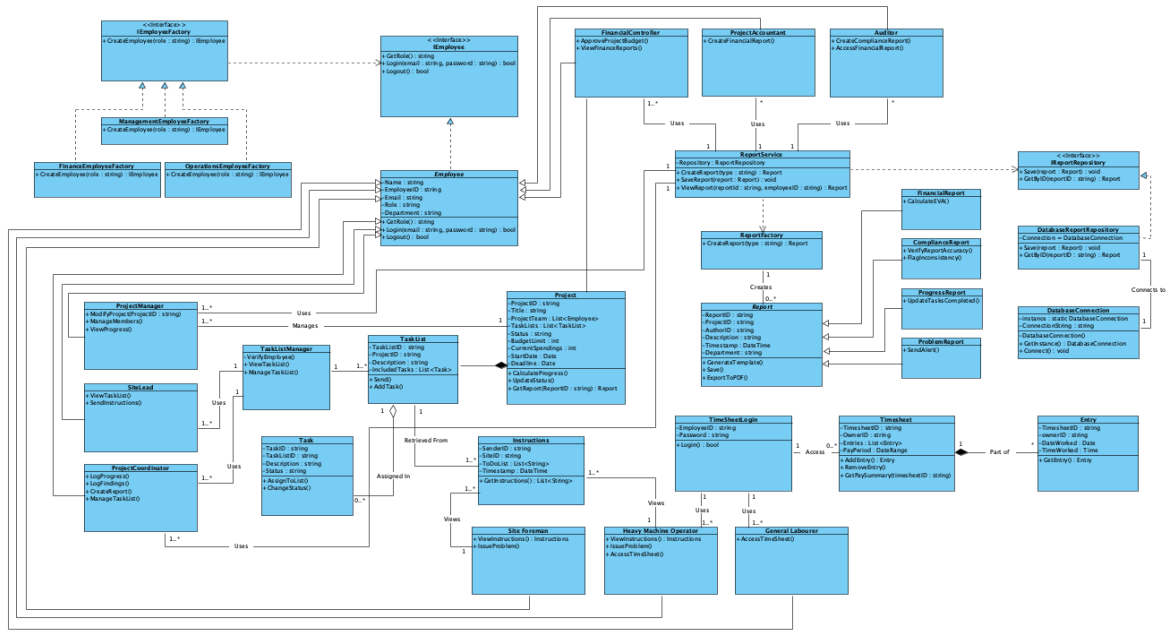


Figure 2: Refined UML Class Diagram

This Class diagram had many changes applied in order to make it into a low coupled system, where each section doesn't rely on another heavily. By implementing abstract classes and interfaces in the correct situations, this diagram didn't contain issues that the previous iteration had. Different departments and roles are now more independent in order to follow the rule of low coupling and high cohesion.

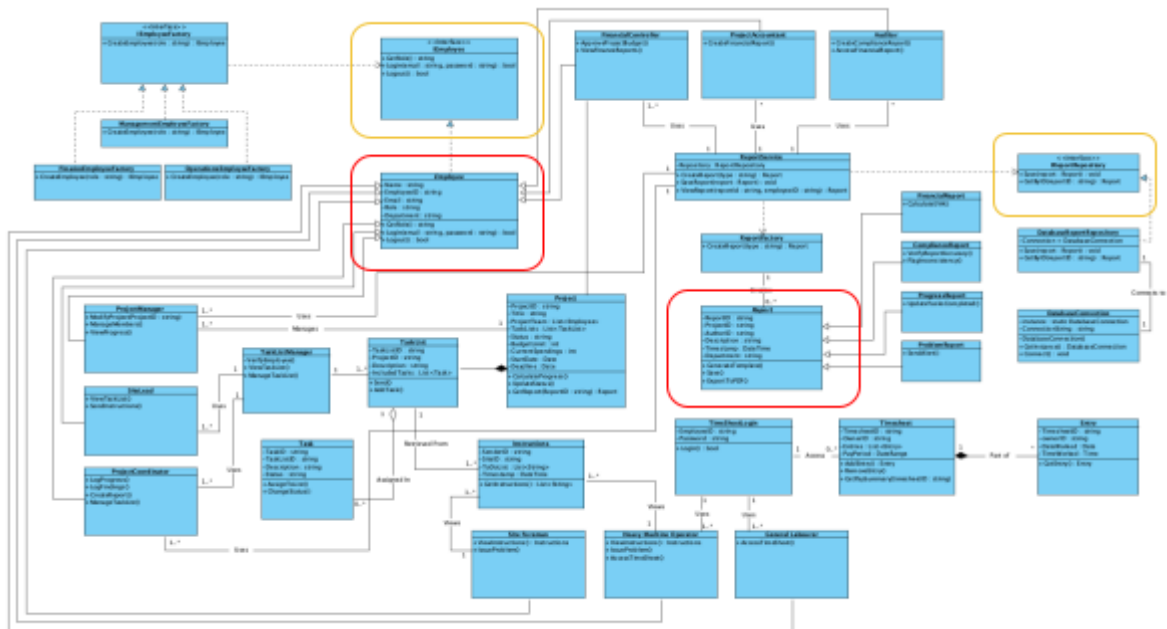


Figure 3: Class Diagram with highlighted abstract + interfaces

Some important changes are the *User* concrete classes becoming *Employee* abstract classes. In the previous iteration, the different employee concrete classes were mapped to a concrete data class. This was changed so that they map to an abstract class instead. The reasoning for this is that the *Employee* class is meant to be a representation of the different types of employees. It should not be an actual implementation which gets initialised in the program. Also abstract classes do not have fully implemented methods, which allows for different employees to have their own methods which can override the inherited method. Therefore making it abstract is the right call.

Looking at *Figure 1*, the interfaces that the different employee classes implemented, were not defined and in some cases were incorrect. They relied on boolean attributes, which have now been removed in the final iteration. Now there is an *IEmployee* interface which every employee sub-class implements. Interfaces should define the behavioural aspects of an object (what it does), not the structural aspect (what it has). Because the boolean was an attribute, it was incorrectly implemented into the model. Looking at *Figure 2*, the new employee interface only contains behavioural blueprints for the employee objects, which follows the correct interface structure.

1.2 Sequence Diagrams